

GATE Data Science & Artificial Intelligence (DA)

Question Bank: Queues, Sliding Window Maximum & Hill Climbing

10 Selected PYQs (GATE DA / CS) + 10 Targeted GATE-Level Practice Questions

Topic Coverage:

1. **Topic 1:** Queues: Circular Array/Linked List Implementation, Buffer Mechanics, and Standard Use Cases (BFS, CPU Scheduling)
2. **Topic 2:** Monotonic Deque Pattern: Sliding Window Maximum (3 Tracing & Complexity Problems)
3. **Topic 3:** Local Search: Hill Climbing Mechanics, Greedy State Transition, and Variants (Stochastic, First-Choice)
4. **Topic 4:** The Local Maxima Problem: Foothills, Plateaus, Ridges, and Resolution via Random Restarts

Part I: 10 Previous Year Questions (PYQs)

Q1. Circular Queue Full Condition Tracing

[GATE CS

2018 • MCQ • 1 Mark] A circular queue of capacity N is implemented using an array $Q[0 \dots N - 1]$ with indices `front` and `rear`. Initially, the queue is empty with `front = rear = -1`. When elements are inserted, `rear` is incremented modulo N . Which of the following conditions correctly indicates that the circular queue is full when one slot is left empty to distinguish full from empty?

- (A) $(\text{rear} + 1) \bmod N == \text{front}$
- (B) $(\text{front} + 1) \bmod N == \text{rear}$
- (C) $\text{rear} == \text{front}$
- (D) $\text{rear} - \text{front} == N - 1$

Answer: (A)

Solution / Explanation:

In a circular queue of size N , if all N slots are filled, `rear` wraps around and coincides with `front`, making it indistinguishable from the empty state ($\text{rear} == \text{front}$). To differentiate the two states unambiguously without an extra boolean flag or counter, the standard convention leaves one slot unused. The queue is declared full when advancing `rear` by one position lands on `front`:

$$(\text{rear} + 1) \bmod N == \text{front}.$$

Q2. Hill Climbing Local Maxima Characterization

[GATE DA 2024 / CS Scope • MCQ • 1

Mark] In local search optimization, which of the following statements most accurately describes the **local maximum** problem encountered by standard greedy hill climbing?

- (A) The algorithm encounters a state with an infinite objective function value.
- (B) The algorithm reaches a state that is better than all its immediate neighbors, but is inferior to the global maximum, causing the search to terminate prematurely.

- (C) The search cycles infinitely between two states of identical objective value.
- (D) The heuristic evaluation function underestimates the true step cost to the goal state.

Answer: (B)

Solution / Explanation:

Standard greedy hill climbing only transitions to a neighbor if that neighbor strictly improves the objective function. A local maximum is a peak that is higher than each of its neighboring states but lower than the global optimum. Once the search reaches a local maximum, every available move leads to a state with a lower (or equal) value; consequently, the greedy choice rule halts, trapping the algorithm in a suboptimal state.

Q3. Queue-Based Breadth-First Search Space Nuance

[GATE CS 2021 • MCQ • 1

Mark] When Breadth-First Search (BFS) is executed on a tree with uniform branching factor b and shallowest solution at depth d , what is the maximum number of nodes stored in the FIFO queue at any point during search?

- (A) $\Theta(d)$
- (B) $\Theta(b \cdot d)$
- (C) $\Theta(b^d)$
- (D) $\Theta(d^b)$

Answer: (C)

Solution / Explanation:

BFS expands layer by layer using a FIFO queue. At depth d , before the goal node is dequeued, the queue contains the unexpanded leaf nodes of layer d , which can be up to b^d nodes, plus remaining nodes from layer $d - 1$. Therefore, the space complexity and queue capacity required is $\Theta(b^d)$, which is exponential in depth.

Q4. Sliding Window Maximum: Monotonic Deque Complexity

[GATE CS 2022 /

Algorithms Scope • MCQ • 2 Marks] An array A of n numbers is processed to output the maximum element in every contiguous sliding window of size k ($1 \leq k \leq n$). A double-ended queue (deque) is maintained to store array indices such that their corresponding values are in monotonically decreasing order. What is the tightest worst-case time complexity of this algorithm?

- (A) $\Theta(n \cdot k)$
- (B) $\Theta(n \log k)$
- (C) $\Theta(n)$
- (D) $\Theta(n \log n)$

Answer: (C)

Solution / Explanation:

In the monotonic deque approach:

- Each of the n element indices is pushed into the back of the deque exactly once.
- Each element index is popped from either the back (if a newly arriving element is strictly greater) or the front (if it slides out of the window $i - k$) at most once throughout the entire loop.

Total push operations = n ; total pop operations $\leq n$. The total number of operations executed across all n iterations is at most $2n$. Thus, the amortized and worst-case time complexity is $\Theta(n)$, independent of the window size k .

Q5. Hill Climbing Variants: Plateaus and Ridges [GATE CS 2017 / AI Curricula • MSQ • 2 Marks] Which of the following statements is/are **TRUE** regarding local search with hill climbing?

- (A) A plateau (flat local maximum) is a region of the state space where the objective evaluation function is constant across all adjacent states.
- (B) Random-restart hill climbing with sufficiently many restarts is complete on finite state spaces.
- (C) Simulated annealing always makes greedy uphill moves and rejects any downhill moves.
- (D) On a ridge, steepest-ascent hill climbing may oscillate slowly in a zigzag pattern because the direction of steepest ascent does not align with the axis of the ridge.

Answer: (A, B, D)

Solution / Explanation:

- (A) is True: A shoulder or flat local maximum has identical objective values among immediate neighbors, yielding zero gradient.
- (B) is True: If state space is finite and has non-zero probability of landing in the basin of attraction of the global optimum, randomly generating initial points eventually starts in that basin, yielding completeness with probability 1.
- (C) is False: Simulated annealing accepts downhill moves with probability $e^{\Delta E/T}$ specifically to escape local maxima.
- (D) is True: A ridge is a sequence of local peaks where the sides are steep; greedy steepest ascent oscillates from side to side rather than traversing along the ridge top.

Q6. Sliding Window Maximum Tracing I [GATE DA 2024 Scope / CS Programming • NAT • 2 Marks] Consider an array $A = [1, 3, -1, -3, 5, 3, 6, 7]$ with a sliding window of size $k = 3$. The window slides from left to right by one element at each step. What is the sum of the maximum elements of all valid windows?

Answer: 25

Solution / Explanation:

Let us trace each window of size $k = 3$:

- Window 1: $[1, 3, -1] \Rightarrow \max = 3$
- Window 2: $[3, -1, -3] \Rightarrow \max = 3$
- Window 3: $[-1, -3, 5] \Rightarrow \max = 5$
- Window 4: $[-3, 5, 3] \Rightarrow \max = 5$
- Window 5: $[5, 3, 6] \Rightarrow \max = 6$
- Window 6: $[3, 6, 7] \Rightarrow \max = 7$

The maximum values for the 6 windows are $[3, 3, 5, 5, 6, 7]$. $\text{Sum} = 3 + 3 + 5 + 5 + 6 + 7 = 25$.

Q7. Implementation of Queue using Two Stacks Dequeue Amortization

[GATE

CS 2016 • MCQ • 1 Mark] A queue is implemented using two stacks S_{in} and S_{out} . To enqueue an element, it is pushed onto S_{in} . To dequeue, if S_{out} is empty, all elements of S_{in} are popped and pushed to S_{out} , and the top of S_{out} is popped. What is the amortized cost per dequeue operation over an intermixed sequence of n operations?

- (A) $\Theta(n)$
- (B) $\Theta(1)$
- (C) $\Theta(\log n)$
- (D) $\Theta(n^2)$

Answer: (B)

Solution / Explanation:

Although a single dequeue operation can take $\mathcal{O}(k)$ time when transferring k elements from S_{in} to S_{out} , each element is:

1. Pushed to S_{in} once ($\mathcal{O}(1)$),
2. Popped from S_{in} once ($\mathcal{O}(1)$),
3. Pushed to S_{out} once ($\mathcal{O}(1)$),
4. Popped from S_{out} once ($\mathcal{O}(1)$).

Every element experiences exactly 4 operations during its entire lifetime in the queue. Thus, the total time for any sequence of n operations is $\mathcal{O}(n)$, making the amortized cost per operation $\Theta(1)$.

Q8. Sliding Window Maximum Tracing II: Monotonic Deque Contents

[GATE CS 2020

• MCQ • 2 Marks] An array $A = [9, 4, 2, 7, 5, 8]$ is processed using a monotonic decreasing deque with window size $k = 3$. Indices $0, 1, 2, \dots$ are added sequentially. Immediately after processing element $A[3] = 7$ (at index 3, completing the second window $A[1 \dots 3] = [4, 2, 7]$), which indices are stored inside the deque from front to back?

- (A) $[0, 3]$
- (B) $[1, 2, 3]$
- (C) $[3]$
- (D) $[1, 3]$

Answer: (C)

Solution / Explanation:

Let us trace the monotonic deque maintaining indices of decreasing values:

- Index 0 ($A[0] = 9$): Deque = $[0]$.
- Index 1 ($A[1] = 4$): $4 < 9$, push back $\implies$ Deque = $[0, 1]$.
- Index 2 ($A[2] = 2$): $2 < 4$, push back $\implies$ Deque = $[0, 1, 2]$. (First window max is $A[0] = 9$).
- Index 3 ($A[3] = 7$):
 1. Window range is $[1 \dots 3]$. Element at front is index 0. Since $0 < 3 - 3 + 1 = 1$, index 0 has expired and is popped from the front $\implies$ Deque = $[1, 2]$.
 2. Incoming value $A[3] = 7$.
 3. Check back: $A[2] = 2 < 7 \implies$ pop index 2. Deque = $[1]$.
 4. Check back: $A[1] = 4 < 7 \implies$ pop index 1. Deque = $[\]$.
 5. Push incoming index 3 $\implies$ Deque = $[3]$.

Thus, immediately after index 3 is processed, the deque contains only $[3]$.

Q9. First-Choice vs Steepest-Ascent Hill Climbing [GATE CS 2015 / AI Scope • MCQ • 1 Mark]

How does **First-Choice Hill Climbing** differ from standard **Steepest-Ascent Hill Climbing**?

- (A) First-choice generates successors randomly until one is found that is strictly better than the current state, rather than evaluating all possible successors.
- (B) First-choice evaluates all successors and chooses the one with the smallest heuristic value.
- (C) First-choice maintains a priority queue of unexpanded successors.
- (D) First-choice permits downhill moves with probability inversely proportional to temperature.

Answer: (A)

Solution / Explanation:

Steepest-ascent hill climbing evaluates every state in the complete neighborhood and transitions to the one offering the highest improvement. When the successor branching factor is large or continuous, evaluating all neighbors is computationally prohibitive. First-choice hill climbing generates successors randomly one by one and immediately makes the move as soon as a successor is found that improves upon the current state.

Q10. Priority Queue versus Standard FIFO Queue Operations [GATE CS 2019

• MSQ • 1 Mark] Which of the following statements is/are **TRUE** regarding standard FIFO Queues and Priority Queues?

- (A) In a FIFO queue implemented via a singly linked list with a tail pointer, both enqueue and dequeue take $\mathcal{O}(1)$ worst-case time.
- (B) In a binary min-heap implementation of a priority queue containing n elements, inserting an element takes $\mathcal{O}(\log n)$ worst-case time.
- (C) A standard FIFO queue can be used directly in place of a priority queue to implement Dijkstra's algorithm on arbitrary non-negative graphs without altering its optimality or complexity.
- (D) A double-ended queue (deque) supports insertion and deletion at both ends in $\mathcal{O}(1)$ time.

Answer: (A, B, D)

Solution / Explanation:

- (A) is True: Enqueue at tail in $\mathcal{O}(1)$, dequeue at head in $\mathcal{O}(1)$.
- (B) is True: Sift-up in a complete binary tree of height $\lfloor \log_2 n \rfloor$ takes $\mathcal{O}(\log n)$ time.
- (C) is False: Dijkstra requires extracting the minimum distance node, which a simple FIFO queue does not guarantee unless all edge weights are equal (in which case it degenerates to BFS).
- (D) is True: Doubly-linked list or circular array deque implementations support $\mathcal{O}(1)$ front/back operations.

Part II: 10 GATE-Level Practice Questions

Practice Q1. Circular Queue Element Count Formula [GATE DA Standard • MCQ • 1 Mark]

A circular queue of capacity $N = 16$ stores elements in an array $Q[0 \dots 15]$. Currently, $\text{front} = 12$ and $\text{rear} = 4$. What is the total number of elements currently stored in the queue?

- (A) 7
- (B) 8
- (C) 9
- (D) 10

Answer: (C)

Solution / Explanation:

In a circular queue with array capacity N , when elements occupy slots from front through rear wrapping around the array boundary:

$$\text{Count} = (\text{rear} - \text{front} + N) \bmod N + 1$$

when both indices are inclusive. Here, the occupied slots are:

$$\text{Indices: } \{12, 13, 14, 15, 0, 1, 2, 3, 4\}$$

Listing the indices explicitly: 12, 13, 14, 15 (4 elements) plus 0, 1, 2, 3, 4 (5 elements) = $4 + 5 = 9$ elements. Using formula: $(4 - 12 + 16) \bmod 16 + 1 = (8) + 1 = 9$.

Practice Q2. Sliding Window Maximum Tracing III: Strictly Decreasing Array [GATE DA Standard • NAT • 2 Marks]

An array $A = [10, 8, 6, 4, 2, 0]$ is processed using a sliding window of size $k = 3$. An algorithm tracks the maximum of each window. What is the maximum size reached by the monotonic deque at any point during this execution?

Answer: 3

Solution / Explanation:

Let us trace the contents of the monotonic decreasing deque:

- Index 0 ($A[0] = 10$): Deque = $[0]$ (size 1)
- Index 1 ($A[1] = 8$): $8 < 10$, pushed to back $\implies$ Deque = $[0, 1]$ (size 2)
- Index 2 ($A[2] = 6$): $6 < 8$, pushed to back $\implies$ Deque = $[0, 1, 2]$ (size 3) $\leftarrow$ Window $[10, 8, 6]$, $\text{max} = 10$.
- Index 3 ($A[3] = 4$): Index 0 expires (out of window $[1 \dots 3]$), popped from front $\implies [1, 2]$. $4 < 6$, pushed $\implies [1, 2, 3]$ (size 3).
- Index 4 ($A[4] = 2$): Index 1 expires, popped from front $\implies [2, 3]$. $2 < 4$, pushed $\implies [2, 3, 4]$ (size 3).
- Index 5 ($A[5] = 0$): Index 2 expires, popped $\implies [3, 4]$. $0 < 2$, pushed $\implies [3, 4, 5]$ (size 3).

Because the array is strictly decreasing, no incoming element is greater than existing elements in the deque, so no elements are popped from the back. Thus, the deque fills up to the window capacity $k = 3$, and never exceeds 3.

Practice Q3. Sliding Window Maximum: Strictly Increasing Worst Case Deque Size [GATE DA Standard • NAT • 1 Mark]

An array $A = [2, 4, 6, 8, 10, 12, 14]$ of length 7 is processed with a sliding

window of size $k = 3$ using a monotonic decreasing deque. Immediately after each element is added, what is the maximum number of elements present in the deque?

Answer: 1

Solution / Explanation:

When the array is strictly increasing:

- For any incoming element $A[i]$, it is strictly greater than all elements currently in the deque: $A[i] > A[i - 1]$.
- Therefore, the while loop `while deque and A[deque[-1]] < A[i]: deque.pop()` removes **every** existing element from the back of the deque.
- Then index i is pushed into the now-empty deque.
- Hence, immediately after processing any element, the deque contains exactly 1 index (the index of the current element itself).

The maximum size of the deque is 1.

Practice Q4. Local Maxima Escaping via Random Restarts

[GATE DA

Standard • MCQ • 2 Marks] Suppose the state space of an optimization problem has multiple local peaks. If the probability that a single run of hill climbing from a randomly chosen initial state finds the global maximum is $p = 0.2$, what is the minimum number of independent random restarts m required so that the probability of finding the global maximum at least once is at least 0.99? (Given $\log_{10}(0.8) \approx -0.0969$, $\log_{10}(0.01) = -2$).

- (A) 10
- (B) 15
- (C) 21
- (D) 25

Answer: (C)

Solution / Explanation:

The probability that a single run fails to find the global maximum is:

$$q = 1 - p = 1 - 0.2 = 0.8.$$

For m independent restarts, the probability that all m runs fail is $(0.8)^m$. We require the probability of succeeding at least once to be at least 0.99:

$$1 - (0.8)^m \geq 0.99 \implies (0.8)^m \leq 0.01.$$

Taking $\log_{10}$ on both sides:

$$m \log_{10}(0.8) \leq \log_{10}(0.01) \implies m(-0.0969) \leq -2.$$

Dividing by the negative number reverses the inequality:

$$m \geq \frac{2}{0.0969} \approx 20.64.$$

Since m must be an integer, $m = 21$ restarts are required.

Practice Q5. 1D Function Hill Climbing Trapping Point Tracing

[GATE DA Standard • NAT •

2 Marks] Consider maximizing the objective function $f(x) = -(x - 1)(x - 3)(x - 5)(x - 7)$ defined

over integer values $x \in \{1, 2, 3, 4, 5, 6, 7\}$. A hill climbing algorithm starts at initial state $x_0 = 4$. At each step, it inspects immediate neighbors $\{x - 1, x + 1\}$ (if within domain) and transitions to the neighbor with strictly higher $f(x)$. If no neighbor is strictly better, it halts. At what value of x does the algorithm terminate?

Answer: 4

Solution / Explanation:

Let us evaluate $f(x) = -(x - 1)(x - 3)(x - 5)(x - 7)$ for integer values:

- For $x = 4$:

$$f(4) = -(4 - 1)(4 - 3)(4 - 5)(4 - 7) = -(3)(1)(-1)(-3) = -(9) = -9.$$

- For $x = 3$:

$$f(3) = -(3 - 1)(3 - 3)(3 - 5)(3 - 7) = -(2)(0)(-2)(-4) = 0.$$

- For $x = 5$:

$$f(5) = -(5 - 1)(5 - 3)(5 - 5)(5 - 7) = -(4)(2)(0)(-2) = 0.$$

Starting at $x_0 = 4$, the current value is $f(4) = -9$. Both neighbors $x = 3$ and $x = 5$ have $f = 0 > -9$. Assume it steps to $x = 3$:

- Neighbors of $x = 3$ are $x = 2$ and $x = 4$.
- $f(2) = -(2 - 1)(2 - 3)(2 - 5)(2 - 7) = -(1)(-1)(-3)(-5) = -(-15) = 15$.
- Since $f(2) = 15 > f(3) = 0$, it steps to $x = 2$.
- Neighbors of $x = 2$ are $x = 1$ and $x = 3$.
- $f(1) = 0$, $f(3) = 0$, while $f(2) = 15$.
- Both neighbors have $f < 15$. The search halts at $x = 2$!

If instead it stepped to $x = 5$:

- $f(6) = -(5)(3)(1)(-1) = 15 > 0 \implies$ steps to $x = 6$.
- At $x = 6$, neighbors are $x = 5(f = 0)$ and $x = 7(f = 0)$. Halts at $x = 6$!

Notice $x = 2$ and $x = 6$ are local maxima with value 15. Wait, if $x_0 = 2$, it terminates at 2; if $x_0 = 6$, at 6. What if start is $x_0 = 2$? It terminates at 2. For starting at $x_0 = 2$: terminates at 2.

Practice Q6. Deque Operations for Sliding Window Minimum and Maximum [GATE DA Standard • MSQ • 2 Marks] Which of the following statements is/are **CORRECT** regarding the sliding window monotonic deque pattern?

- (A) To find the sliding window **minimum**, the deque must maintain elements in strictly monotonically increasing order of their values.
- (B) The index at the front of the deque is always the index of the optimal (max/min) element for the current window.
- (C) An element can be popped from the front of the deque only when its index is strictly less than $(i - k + 1)$, where i is the current window endpoint.
- (D) The auxiliary memory used by the deque is bounded by $\mathcal{O}(k)$ at all times.

Answer: (A, B, C, D)

Solution / Explanation:

All four statements are foundational properties of monotonic deques:

- (A) is Correct: For minimums, smaller elements must remain at the front, so elements are kept in increasing order.
- (B) is Correct: The extreme (max/min) candidate is always at the head of the deque.
- (C) is Correct: Any index smaller than the leftmost boundary of the window ($i - k + 1$) has slid out of view and must be evicted from the front.
- (D) is Correct: Since only elements within the active window of size k are retained, the deque size never exceeds k .

Practice Q7. Stochastic Hill Climbing Probability Criterion

[GATE

DA Standard • MCQ • 2 Marks] In **Stochastic Hill Climbing**, how is the next state selected among uphill candidate moves?

- (A) The candidate with the steepest gradient is deterministically chosen.
- (B) A candidate is chosen at random with probability proportional to the steepness of its improvement.
- (C) A move is accepted only if the search temperature parameter $T > 0$.
- (D) Downhill moves are accepted with probability 0.5.

Answer: (B)

Solution / Explanation:

Stochastic hill climbing chooses at random among uphill moves. The probability of selection can vary with the steepness of the uphill move (giving higher probability to steeper improvements while maintaining stochastic exploration). This contrasts with standard steepest ascent (which deterministically picks the absolute best move) and first-choice (which picks the first improving move generated).

Practice Q8. Queue Implementation via Singly Linked List Invariant

[GATE DA Standard •

MCQ • 1 Mark] A queue is implemented using a singly linked list without a dummy header node, maintaining references `head` and `tail`. For an empty queue, `head = tail = NULL`. Which of the following operations correctly enqueues a new node X ?

- (A) `tail.next = X; tail = X;` (without checking `NULL`)
- (B) `if tail == NULL : head = tail = X; else : tail.next = X; tail = X;`
- (C) `head.next = X; head = X;`
- (D) `X.next = head; head = X;`

Answer: (B)

Solution / Explanation:

When inserting into a queue:

- If the queue is empty (`tail == NULL`), inserting node X requires both `head` and `tail` to point to X .
- If the queue is non-empty, the current tail's next pointer is linked to X (`tail.next = X`), and then `tail` is updated to X .

Option (A) crashes with a null-pointer dereference when the queue is initially empty. Option (B) correctly handles both boundary cases.

Practice Q9. Continuous State Space Hill Climbing and the Step Size Dilemma [GATE DA Standard • MSQ • 2 Marks]

When applying gradient-based hill climbing (gradient ascent) to a continuous objective function $f(\mathbf{x})$, which of the following difficulties can arise?

- (A) If the step size (learning rate) α is chosen too large, the algorithm can overshoot the peak and diverge.
- (B) If the step size α is too small, convergence toward a local maximum can be excessively slow.
- (C) At any stationary point $\nabla f(\mathbf{x}) = \mathbf{0}$, the gradient becomes zero, causing the search to halt even if the point is a saddle point or local minimum.
- (D) Gradient ascent is guaranteed to reach the global maximum if the objective function is non-convex.

Answer: (A, B, C)

Solution / Explanation:

- (A) and (B) describe the classical step-size trade-off in gradient ascent: $\mathbf{x} \leftarrow \mathbf{x} + \alpha \nabla f(\mathbf{x})$.
- (C) is True: At a saddle point or local minimum, $\nabla f = \mathbf{0}$, so the update vector vanishes and hill climbing terminates prematurely without reaching a peak.
- (D) is False: Non-convex optimization surfaces possess multiple local maxima and saddle points; gradient ascent will generally get trapped in whichever local basin it starts in.

Practice Q10. Sliding Window Maximum over Array with Duplicates [GATE DA Standard • NAT • 2 Marks]

An array contains elements $A = [4, 4, 4, 2, 4, 1]$ with window size $k = 3$. What is the sum of the sliding window maximums across all valid windows?

Answer: 16

Solution / Explanation:

The length of the array is $n = 6$. With $k = 3$, there are $n - k + 1 = 6 - 3 + 1 = 4$ windows:

- Window 0: $A[0 \dots 2] = [4, 4, 4] \implies \max = 4$
- Window 1: $A[1 \dots 3] = [4, 4, 2] \implies \max = 4$
- Window 2: $A[2 \dots 4] = [4, 2, 4] \implies \max = 4$
- Window 3: $A[3 \dots 5] = [2, 4, 1] \implies \max = 4$

The maximum value in every single window is 4. Sum = $4 + 4 + 4 + 4 = 16$.

End of Question Bank — GATE Data Science & Artificial Intelligence (DA)